

CIVICSENSE AI

AI-Powered Civic Complaint Detection, Generation & Routing Platform

PROJECT TEAM

Person 1: Arpit

Person 2: Arnav Yadav

Core Idea

A citizen can capture or upload a photo of a civic problem with location context. CivicSense AI identifies the issue, estimates severity, determines the relevant civic context and authority, retrieves supporting civic information, generates a formal complaint and prepares it for submission and tracking.

Technology Focus

Python • FastAPI • Next.js • React • TypeScript • Tailwind CSS • PostgreSQL
• Redis • PyTorch • YOLO • OpenCV • Scikit-learn • Transformers • BGE •
Qdrant • Ollama • Docker • Git/GitHub

The responsibility split is designed so both members perform substantial AI work while keeping ownership clear and integration straightforward.

1. What is CivicSense AI?

CivicSense AI is an AI-driven civic complaint platform designed to simplify how citizens report real-world public infrastructure and civic problems. Instead of requiring citizens to identify the correct department, understand complaint procedures and write a formal complaint manually, the platform uses AI to automate the intelligence-heavy portions of this process from a photograph and location context.

Typical citizen experience:

1. Capture/upload a photograph of the civic issue.
2. Provide or allow access to the issue location/GPS coordinates.
3. AI analyzes the photograph and identifies the civic problem.
4. AI estimates severity and creates structured issue information.
5. Location intelligence determines the relevant geographic jurisdiction.
6. Civic authority intelligence identifies the responsible authority/department using verified civic data.
7. RAG retrieves relevant official civic information.
8. A local LLM generates a professional complaint from verified information.
9. The citizen reviews the complaint.
10. The application submits the complaint and provides tracking/follow-up.

Project objective

Reduce the friction between seeing a civic problem and reporting it correctly by combining visual understanding, civic knowledge, geographic intelligence and automated complaint preparation.

The platform assists the citizen rather than replacing the citizen or the authority. Official authority/contact information is maintained as verified data and is not invented by an LLM.

2. Major Project Features

Feature	Purpose
AI Civic Issue Detection	Detects potholes, garbage accumulation, broken streetlights, water leakage, sewage overflow, road damage, damaged footpaths, fallen trees, illegal dumping and other supported civic issues.
Severity Intelligence	Estimates LOW, MEDIUM, HIGH or CRITICAL severity from visible damage, affected area, obstruction and defined signals.
Location Intelligence	Converts GPS coordinates into usable geographic context such as address, city, district and available ward/zone information.
Authority Routing	Maps issue type + location/jurisdiction to the responsible authority and department using verified civic information.
Civic RAG	Retrieves relevant information from official civic documents using BGE embeddings and Qdrant.
AI Complaint Generation	Generates a structured subject, body, priority and requested action using verified issue, location, authority and retrieved context.
Duplicate Detection	Identifies potentially duplicate complaints using geographic proximity, issue similarity, image similarity and time.
Civic Hotspots	Uses spatial clustering to identify areas with concentrated civic complaints.
Trend Analysis	Analyzes complaint category, severity, location and time to identify increasing/decreasing or recurring civic problems.

Feature	Purpose
Complaint Tracking	The application layer stores, submits and tracks complaints after AI preparation.

3. Technology Stack

Layer	Technology
Frontend	Next.js, React, TypeScript, Tailwind CSS
Main Backend	Python, FastAPI
Database	PostgreSQL
Caching / Queues	Redis
Computer Vision	PyTorch, Ultralytics YOLO, OpenCV, NumPy
Machine Learning	Scikit-learn
NLP / LLM	Transformers, Ollama, local LLM
Embeddings	BGE
Vector Database	Qdrant
Maps / Location	Leaflet or Mapbox, reverse geocoding
Containerization	Docker
Version Control	Git / GitHub

Architecture principle

The project uses one consistent application stack. The main FastAPI backend handles the product/application layer, while AI functionality can be exposed through a dedicated FastAPI AI service. The application backend consumes structured AI results without depending on model internals.

4. End-to-End Architecture

CITIZEN → NEXT.JS FRONTEND → MAIN FASTAPI → AI SERVICE → VISION + SEVERITY + LOCATION + AUTHORITY + RAG + LLM → STRUCTURED COMPLAINT → USER REVIEW → SUBMISSION → AUTHORITY → TRACKING

Core AI flow:

Photograph → Image preprocessing → YOLO civic issue detection → Issue classification + confidence → Severity intelligence → GPS/location intelligence → Authority/department routing → Official civic knowledge retrieval → RAG context + LLM reasoning → Structured complaint → Validation → Application backend

Critical data rule

The LLM must not be treated as the source of truth for official authority names, email addresses or contact details. Those values should come from verified civic datasets/official sources.

5. Person 1 — Arpit

Role: AI Intelligence & Core Intelligence Engineer

Arpit owns the core AI layer that understands the photograph, evaluates the civic problem and provides the intelligence required by the rest of the platform. Arpit also owns the central AI architecture and its evaluation.

A. Computer Vision & Image Intelligence

- Build image intake and preprocessing using OpenCV.
- Validate image format, resolution, corruption, blur and basic quality.
- Prepare images for model training/inference.
- Develop the YOLO-based civic issue detection system.
- Prepare datasets, labels and augmentation where appropriate.
- Train, validate and evaluate the detection model.
- Implement the initial civic issue classes.
- Handle multiple detected issues and primary/secondary issue selection.
- Produce confidence scores and structured detection output.
- Measure precision, recall, mAP@50, mAP@50:95, per-class performance and inference time.

B. Severity Intelligence

- Design severity methodology and features.
- Estimate severity from visible damage, affected area, obstruction and defined signals.
- Produce LOW, MEDIUM, HIGH or CRITICAL severity.
- Generate a severity score and supporting factors.
- Test severity consistency and define fallback behavior.

C. Core AI Decision Engine

- Combine vision, severity, location, authority and retrieved civic context into a unified structured civic case.
- Define internal AI data contracts.
- Implement decision/validation logic before complaint generation.
- Ensure downstream components receive predictable JSON.

D. RAG / Civic Knowledge Intelligence

- Design RAG architecture.
- Process documents into clean chunks.
- Generate BGE embeddings.
- Store/query embeddings in Qdrant.
- Implement semantic retrieval and top-k context selection.
- Connect retrieved context to the local LLM.
- Evaluate retrieval relevance, context quality and grounding.

- Implement safeguards for missing or unsupported information.

E. LLM & Prompt Intelligence

- Integrate Ollama/local LLM inference.
- Design system prompts and prompt templates.
- Define structured JSON outputs.
- Build prompt/context assembly.
- Implement output validation and guardrails.
- Prevent unsupported claims and hallucinated civic facts.
- Design fallback behavior when the model cannot confidently answer.

F. AI Evaluation & Model Quality

- Create AI evaluation datasets and test cases.
- Evaluate vision models, RAG retrieval and LLM outputs.
- Track hallucination, grounding, JSON validity and complaint completeness.
- Document model versions, experiments, metrics and limitations.
- Optimize inference where practical.

6. Person 2 — Arnav Yadav

Role: AI Application & Civic Intelligence Engineer

Arnav owns the civic intelligence systems surrounding the core AI. His work is also AI/ML-heavy: geographic intelligence, authority routing, duplicate detection, spatial clustering, trend analysis and AI-facing service integration.

A. Location Intelligence

- Integrate latitude/longitude input with reverse geocoding.
- Convert coordinates into address, city, district and available ward/zone information.
- Normalize geographic fields for downstream AI and database use.
- Handle missing, inaccurate or ambiguous location data.
- Build geographic utilities required by authority routing and duplicate detection.

B. Authority & Department Intelligence

- Build and maintain the structured civic authority dataset.
- Map issue categories to responsible departments.
- Map geographic jurisdictions to authorities.
- Combine issue type + jurisdiction + department responsibility.
- Return responsible authority and department with confidence/validation metadata.
- Store source and verification date for official contact information.
- Implement authority validation so unsupported contacts are not passed to the complaint system.

C. Civic Knowledge Data Preparation

- Collect and structure official civic documents and authority information.
- Maintain department responsibility rules.
- Prepare structured records for the RAG system.
- Track source, jurisdiction and last verification date.
- Coordinate with Arpit when new or updated civic knowledge enters the RAG pipeline.

D. Duplicate Complaint Detection

- Design duplicate detection logic/model.
- Compare GPS proximity between complaints.
- Compare issue categories and structured metadata.
- Use image similarity where appropriate.
- Use description/text similarity where appropriate.
- Consider time difference between reports.
- Produce a potential-duplicate score and reference to an existing complaint.
- Test false-positive and false-negative cases.

E. Civic Hotspot Detection

- Analyze complaint coordinates to identify geographic clusters.

- Implement spatial clustering such as DBSCAN where appropriate.
- Identify high-density issue areas.
- Separate hotspots by issue category and severity.
- Prepare structured hotspot results for dashboards/analytics.

F. Civic Trend Intelligence

- Analyze complaints across time, category, severity and location.
- Identify increasing/decreasing trends.
- Identify recurring civic problems.
- Generate 7-day, 30-day or other time-window patterns.
- Prepare AI/ML-derived analytics for the application dashboard.

G. AI Service Integration & Testing

- Implement/maintain AI-facing endpoints for Arnav's civic intelligence modules.
- Integrate location, authority, duplicate and analytics services with the AI service.
- Create integration tests for AI service communication.
- Validate request/response schemas.
- Document interfaces consumed by the main FastAPI backend.
- Coordinate integration without modifying Arpit's internal model implementations.

7. Shared Work & Ownership Boundaries

Both members work on AI, but each owns a different layer. These boundaries prevent code conflicts and duplicated implementation.

Area	Ownership
Arpit owns	Core vision, YOLO, severity intelligence, core decision engine, RAG architecture, BGE embeddings, Qdrant retrieval, Ollama/LLM reasoning, prompt architecture and AI evaluation.
Arnav owns	Location intelligence, authority routing, civic authority data, duplicate detection, spatial clustering/hotspots, civic trend analytics and AI service integration for these modules.
Shared	Python, FastAPI conventions, Docker, Git/GitHub, testing standards, API contracts, documentation, integration testing and end-to-end verification.
Main application	The application layer handles user accounts, frontend, complaint lifecycle, persistence, submission, notifications and tracking. AI modules return structured results to this layer.

What Arpit does NOT own

- Frontend pages/UI as the primary owner.
- Authority contact data as an unverified source of truth.
- Complaint tracking/lifecycle implementation.
- Duplicate/hotspot/trend application analytics owned by Arnav.

What Arnav does NOT own

- YOLO architecture/training implementation.
- Core RAG architecture and embedding pipeline.
- Core prompt architecture and LLM guardrails.
- Internal implementation of Arpit's core vision/severity models.

8. AI Service Contract

The cleanest integration point is a structured AI service. The main application does not need to know how the models work internally.

Core analysis request:

```
POST /ai/analyze
image_url
latitude
longitude
user_description (optional)
```

Example structured response:

```
{
  "issue": {"type": "pothole", "confidence": 0.94},
  "severity": {"level": "high", "score": 0.86},
  "location": {"address": "...", "city": "..."},
  "authority": {"name": "...", "department": "..."},
  "complaint": {"subject": "...", "body": "...", "priority": "high"}
}
```

Integration rule

Internal model changes are allowed as long as the agreed API contract remains stable. This lets both members develop independently and makes final integration predictable.

9. Development Roadmap

Phase	Name	Main Work
Phase 1	Foundation	Repository structure, Python environments, FastAPI services, API contracts, basic image intake and initial civic datasets.
Phase 2	Computer Vision	Dataset preparation, YOLO training, evaluation, issue detection and classification.
Phase 3	Severity	Severity methodology, scoring, validation and structured output.
Phase 4	Location Intelligence	Reverse geocoding, geographic normalization, jurisdiction mapping.
Phase 5	Authority Intelligence	Authority dataset, issue-to-department mapping, jurisdiction rules and validation.
Phase 6	RAG	Document processing, BGE embeddings, Qdrant retrieval, grounding and evaluation.
Phase 7	Complaint Intelligence	Prompt architecture, Ollama, structured complaint generation and validation.
Phase 8	Advanced AI	Duplicate detection, image/text similarity, hotspots, clustering and trend intelligence.
Phase 9	Integration	End-to-end photo → AI → civic intelligence → complaint → user review → submission → tracking.

10. Final Deliverables

Arpit's primary AI deliverables

- Image preprocessing and quality pipeline.
- Trained/evaluated civic issue detection model.
- Issue classification and confidence system.
- Severity intelligence module.
- Core AI decision engine.
- BGE embedding + Qdrant retrieval pipeline.
- RAG orchestration and evaluation.
- Ollama/local LLM integration.
- Prompt and structured-output framework.
- AI evaluation suite and model documentation.

Arnav's primary AI deliverables

- Location intelligence and geographic normalization.
- Authority/department routing engine.
- Verified civic authority dataset.
- Duplicate complaint detection system.
- Image/text/geographic similarity logic as required.
- Civic hotspot and spatial clustering engine.
- Civic trend analysis engine.

- AI service endpoints/integration for these modules.
- Integration tests and AI-service documentation.

Combined project deliverables

- Stable AI API contract.
- End-to-end AI analysis pipeline.
- Structured complaint generation.
- Verified authority routing.
- Duplicate/hotspot/trend analytics.
- Dockerized development environment.
- Git/GitHub repository with documented ownership.
- End-to-end tests and project documentation.

11. Final CivicSense AI Workflow

CITIZEN → PHOTO + GPS → IMAGE QUALITY CHECK (Arpit) → CIVIC ISSUE DETECTION (Arpit) → ISSUE CLASSIFICATION (Arpit) → SEVERITY (Arpit) → LOCATION INTELLIGENCE (Arnav) → AUTHORITY/DEPARTMENT ROUTING (Arnav) → CIVIC KNOWLEDGE RETRIEVAL (Arpit + Arnav's structured data) → AI DECISION ENGINE (Arpit) → COMPLAINT GENERATION (Arpit) → DUPLICATE CHECK (Arnav) → USER REVIEW → SUBMISSION → AUTHORITY → TRACKING

Final responsibility model

Arpit: makes the system understand the image and reason over civic information. Arnav: makes that intelligence geographically and operationally useful through authority routing, duplicate detection, spatial intelligence and civic analytics. Together, they form the complete CivicSense AI intelligence pipeline.